Testing & Test Cases

Testing: 依据需求确定 expected result，再观察 actual result 是否一致；
发现差异后保留证据，帮助定位原因。通过只说明已检查的情况符合预期。
Test case: 写清前置状态、输入、步骤和预期，让同一检查能被别人复现。
Assertion: 把预期写成会失败的代码判断，例如提示正确、数据确实保存。

Unit 验证局部规则，如必填校验；Integration 验证组件协作，如接口写入数据。
System 检查完整系统行为；Acceptance 根据业务验收条件判断是否满足需求。

Static: 不运行程序，评审需求或代码；Dynamic: 运行程序，观察实际结果。

Functional 检查功能结果；Non-functional 检查性能、安全和可靠性等质量属性。

Given



假设 name 是必填字段。
留空，其余字段合法；
用户已登录、允许提交，
避免其他错误干扰结果。

```
name = ""
```

When



点击 Submit 提交表单，
触发实际的校验流程。
等待校验结果出现，
再读取提示和保存结果。

```
submit()
```

Then



应显示 name 的必填提示，
并拒绝保存这条记录。
同时检查提示和未保存，
验证完整的业务预期。

```
error == REQUIRED
```

Automation Testing

Automation：工具执行预先定义的检查，测试代码决定 expected result。
适合反复验证稳定规则；未知问题探索和体验判断仍需人工。

01 Prepare

为每条 case 设置环境、用户与初始数据。
例：创建独立记录，明确开始时的状态。
fixture 封装准备，避免依赖别的用例。
结束后清理数据，让每次运行可复现。

02 Execute

Steps 调用 API Client / Page Object。
将点击、请求、读取封装成可复用方法。
UI 等待元素可用或状态更新，并设超时。
固定 sleep 可能过短，也会浪费时间。

03 Assert

expected 来自需求，断言失败让 case 失败。
API 核对状态码、body 和业务状态。
UI 读取目标记录，核对保存后的结果。
例：有成功提示，但未保存，仍应失败。
只检查没有异常，会漏掉业务结果错误。

04 Report

Serenity 记录场景、失败步骤和相关证据。
请求 / 响应或截图帮助还原实际结果。
先区分产品、脚本、数据或环境问题。
flaky test 指同一检查有时通过、有时失败。
修正数据与等待条件，retry 不能代替分析。

Regression Testing



变更后，检查应该保持的行为是否仍然正常。

Triggers: feature / bug fix / refactor / config

Regression

检查变更是否破坏原本正常的行为。
例：修复空值校验后，合法输入仍能保存。
范围包含同一模块、共享规则和调用方。

Confirmation / Retest

重跑发现原缺陷的用例，确认它已修复。
例：空值曾被错误保存，修复后应被拒绝。
这能确认原问题，却不能覆盖其他影响。

Smoke

用少量关键路径判断构建是否基本可用。
例：能登录、打开页面并完成主要操作。
通过后再跑广泛回归；通过不代表全正确。

Purpose x Execution

Purpose	Manual	Automated
Regression	人工重测	自动重跑
New checks	人工验证	自动验证

Regression 是目的，Automation 是方式。
例：修改校验逻辑后重测成功 / 拒绝。
根据变更影响，选择相关或完整回归集合。

Test Design & Regression Scope

设计用例先看规则与风险，再决定数据和执行层级。
每条场景独立运行，结果要有明确的判断依据。

Boundary / 边界

边界是合法 / 非法结果发生切换的位置。
假设数量 1–100：测 0/1/2、99/100/101。
检查边界被接受，越界被拒绝，例如：

```
assertFalse(isValid(101));
```

Conditions / 条件组合

Decision Table 将条件组合映射到预期。
例：登录且有权限时允许，否则应拒绝。
每个组合写清结果，避免只测试成功路径。
Equivalence 将同类输入分组，选代表值。

State / 状态迁移

同一操作是否允许，可能取决于当前状态。
例：假设 Draft 可编辑，提交后禁止编辑。
既断言合法操作后的目标状态，也检查
非法操作被拒绝、原状态没有被改动。
具体转换与限制必须依据实际需求。

Impact / 回归范围

先找改动模块，以及使用它的共享流程。
Unit 快速检查规则；API 检查接口与权限。
少量 UI / E2E 验证完整用户路径。
修复校验时，同时重测合法与非法输入。
风险与影响决定相关集合或完整回归。

Serenity BDD Architecture

BDD：讨论例子，写成行为场景，再接到测试代码。
Discovery / Formulation / Automation

```
Scenario: Reject an empty name
  Given all other form fields are valid
  And the name field is empty
  When I submit the form
  Then a required-field error is shown
  And the submission is not saved
```

Cucumber 将场景句子匹配到 Java 方法。
Steps 调用操作层、页面对象或 API 客户端。
Page Object 集中维护定位、等待与读取，
页面改动时可复用原有业务场景。
Assertions 根据需求比较实际与预期；
Serenity 记录过程，预期仍由我们定义。

Maven + JUnit Platform 启动测试；CI 用 tags 选择 smoke / regression。





官方计算器示例（界面随版本变化）：步骤、截图、结果与耗时；截图频率依配置。

NPA Sign-off Example

Case: 检查 Unit A sign-off 是否真正保存。
Given: 独立的 Pre-NPA proposal;
Unit A 为 Pending，用户有该 unit 审批权限。
When: 提交该记录的 Unit A sign-off。
用 proposalId 定位记录，避免操作其他数据。

Then: 重新打开同一 proposal，读取 Unit A 状态。
Assertion 将实际状态与 SIGNED_OFF 比较;
这样确认的是持久化结果，成功提示还不够。
改 sign-off 实现后，重跑此 case 做 regression，
再按变更影响补充权限和其他相关场景。

示意片段：方法、权限和状态名称按实际项目实现。

Java Step Definitions

```
@When("I sign off for Unit A")
public void signOff() {
  actions.submitFor("Unit A");
}

@Then("Unit A is signed off")
public void checkResult() {
  page.reopen(proposalId);
  assertEquals(SIGNED_OFF,
      page.statusFor("Unit A"));
}
```

感谢欧阳的指导与帮助

Presentation by Susan Huishan Bi